

```
In [1]: import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: file_path = r"D:\新建文件夹 (5)\大三\数据分析及应用\无锡地铁坐标.csv"

# 从CSV文件中加载坐标信息
coordinates_df = pd.read_csv(file_path)
```

```
In [3]: df = pd.read_csv('D:/新建文件夹 (5)/大三/数据分析及应用/无锡地铁.csv')
df
```

```
Out[3]:
```

|  | Unnamed: 0 | 前一站 | 后一站      | 地铁线 | 时间(分钟) |
|--|------------|-----|----------|-----|--------|
|  | 0          | 1   | 堰桥 锡北运河  | 1   | 2.0    |
|  | 1          | 2   | 锡北运河 西漳  | 1   | 3.0    |
|  | 2          | 3   | 西漳 天一    | 1   | 2.0    |
|  | 3          | 4   | 天一 刘潭    | 1   | 2.0    |
|  | 4          | 5   | 刘潭 庄前    | 1   | 3.0    |
|  | ...        | ... | ...      | ... | ...    |
|  | 161        | 162 | 周新苑 五湖大道 | 4   | 2.0    |
|  | 162        | 163 | 市民中心 周新苑 | 4   | 2.0    |
|  | 163        | 164 | 吴都路 市民中心 | 4   | 3.0    |
|  | 164        | 165 | 丰润道 吴都路  | 4   | 3.0    |
|  | 165        | 166 | 博览中心 丰润道 | 4   | 2.0    |

166 rows × 5 columns

```
In [4]: G = nx.DiGraph()
for idx, row in df.iterrows(): # 遍历表格的每一行
    G.add_edge(row['前一站'], row['后一站'], line=row['地铁线'], time=row['时间(分钟)'])
```

```
In [5]: len(G.nodes())
```

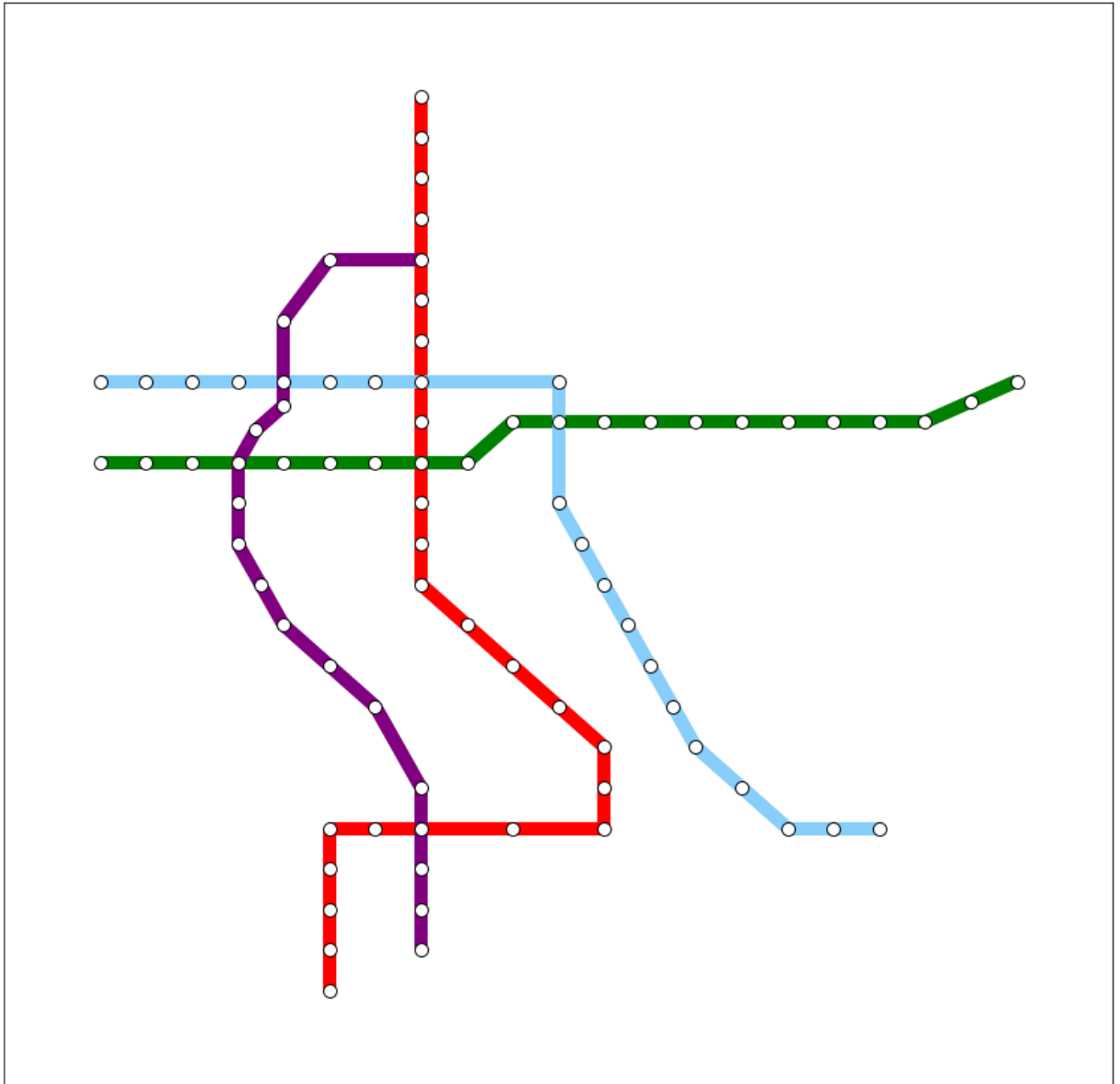
```
Out[5]: 80
```

```
In [6]: for idx, row in coordinates_df.iterrows():
    node = row['节点名称']
    x = row['横坐标']
    y = row['纵坐标']
    nx.set_node_attributes(G, {node: (x, y)}, "pos")
```

```
In [7]: colors = ['r', 'g', 'lightskyblue', 'purple']
lines = set(nx.get_edge_attributes(G, 'line').values())
pos = nx.get_node_attributes(G, 'pos')
plt.figure(figsize=(15,15))
```

```
# 为每条地铁线绘制对应的边，并使用不同颜色
for i, line in enumerate(lines):
    edges = [(u, v) for (u, v, d) in G.edges(data=True) if d['line'] == line]
    nx.draw_networkx_edges(G, pos, edgelist=edges, edge_color=colors[i], width=10.0,
    nx.draw_networkx_nodes(G, pos, node_size=100, node_color='white', edgecolors='black')
```

Out[7]: <matplotlib.collections.PathCollection at 0x270d2d2ae80>



In [8]: `nx.has_path(G, source='江南大学', target='云林')`

Out[8]: True

In [17]: `nx.shortest_path(G, source='江南大学', target='吴桥', weight='time')`

Out[17]:

- '江南大学',
- '文化宫',
- '市民中心',
- '周新苑',
- '五湖大道',
- '大剧院',
- '蠡湖大桥',
- '夏家边',
- '体育中心',
- '西园弄',

```
'河埭口',  
'青山湾荣院',  
'惠山古镇',  
'盛岸',  
'吴桥']
```

```
In [18]: nx.shortest_path_length(G, source='江南大学', target='吴桥', weight='time')
```

```
Out[18]: 34.0
```

```
In [19]: A_station = '江南大学'  
B_station = '吴桥'  
  
# 获取最短路径  
shortest_path_list = nx.shortest_path(G, source=A_station, target=B_station, weight='time')  
  
for i in range(len(shortest_path_list)-1):  
    previous_station = shortest_path_list[i]  
    next_station = shortest_path_list[i+1]  
    line_id = G.edges[(previous_station, next_station)][ 'line' ] # 地铁线编号  
    time = G.edges[(previous_station, next_station)][ 'time' ] # 时间  
    print(' {}--->{} {}号线 {}分钟'.format(previous_station, next_station, line_id, time))  
  
# 最短路径长度  
print(' 共计 {} 分钟'.format(nx.shortest_path_length(G, source=A_station, target=B_station, weight='time')))
```

```
江南大学--->文化宫 1号线 2.0分钟  
文化宫--->市民中心 1号线 3.0分钟  
市民中心--->周新苑 4号线 2.0分钟  
周新苑--->五湖大道 4号线 2.0分钟  
五湖大道--->大剧院 4号线 2.0分钟  
大剧院--->蠡湖大桥 4号线 2.0分钟  
蠡湖大桥--->夏家边 4号线 3.0分钟  
夏家边--->体育中心 4号线 3.0分钟  
体育中心--->西园弄 4号线 3.0分钟  
西园弄--->河埭口 4号线 2.0分钟  
河埭口--->青山湾荣院 4号线 2.0分钟  
青山湾荣院--->惠山古镇 4号线 2.0分钟  
惠山古镇--->盛岸 4号线 3.0分钟  
盛岸--->吴桥 3号线 3.0分钟  
共计 34.0 分钟
```

```
In [22]: import networkx as nx  
import matplotlib.pyplot as plt  
import matplotlib.colors as mcolors  
  
# for Notebook  
%matplotlib inline
```

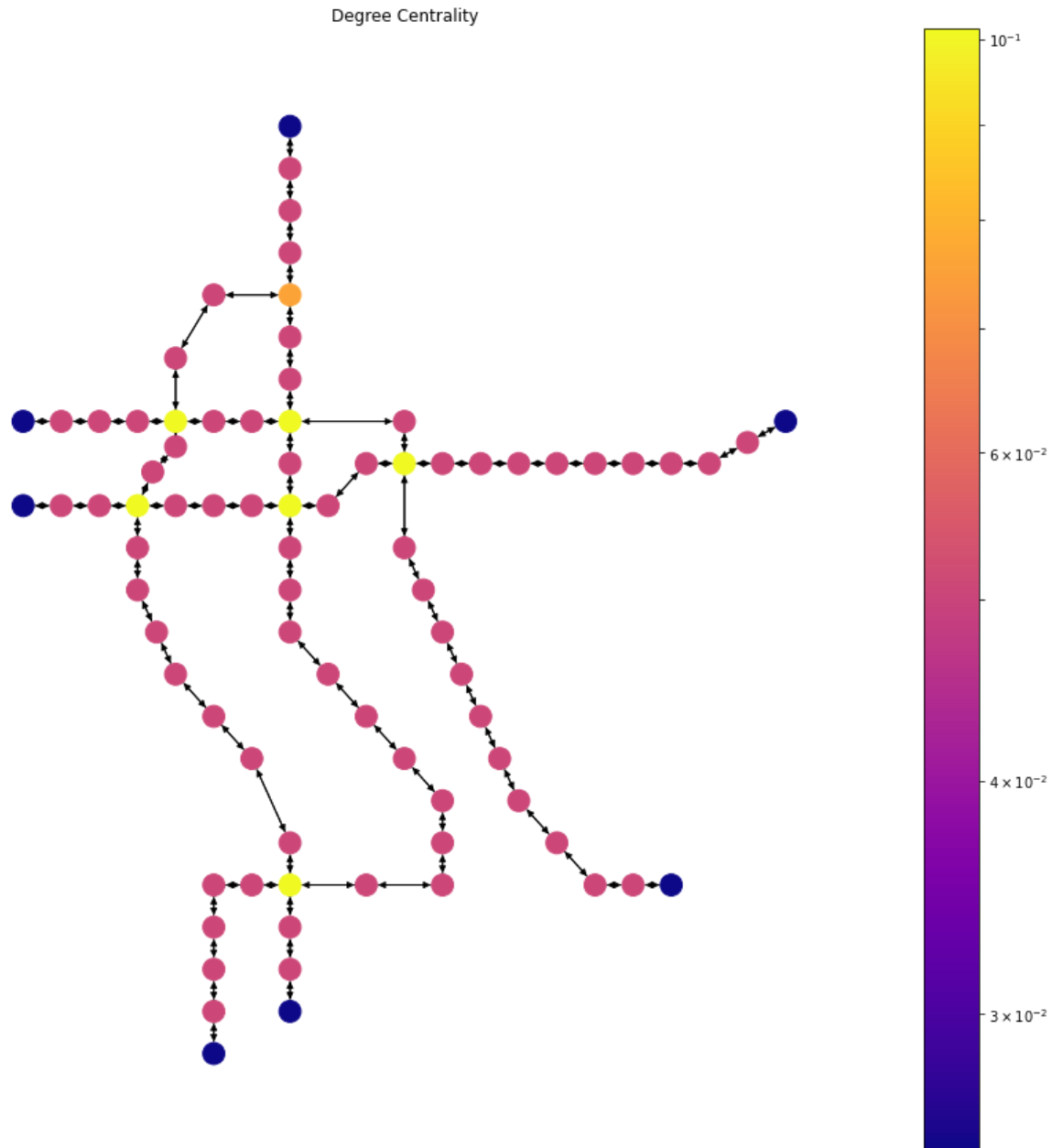
```
In [21]: def draw(G, pos, measures, measure_name):  
  
    nodes = nx.draw_networkx_nodes(G, pos, node_size=250, cmap=plt.cm.plasma,  
                                   node_color=list(measures.values()),  
                                   nodelist=measures.keys())  
    nodes.set_norm(mcolors.SymLogNorm(linthresh=0.01, linscale=1, base=10))  
    # labels = nx.draw_networkx_labels(G, pos)  
    edges = nx.draw_networkx_edges(G, pos)  
  
    plt.title(measure_name)  
    plt.colorbar(nodes)
```

```
plt.axis('off')
plt.show()

def dict_sort_by_value(dict_input):
    """
    输入字典，输出按值排序的字典
    """
    return sorted(dict_input.items(),key=lambda x : x[1], reverse=True)
```

In [23]:

```
plt.figure(figsize=(15,15))
draw(G, pos, nx.degree_centrality(G), 'Degree Centrality')
```



In [24]:

```
import networkx as nx

# 计算节点的特征向量中心性
centrality = nx.eigenvector_centrality(G, max_iter=1000, tol=1e-6)

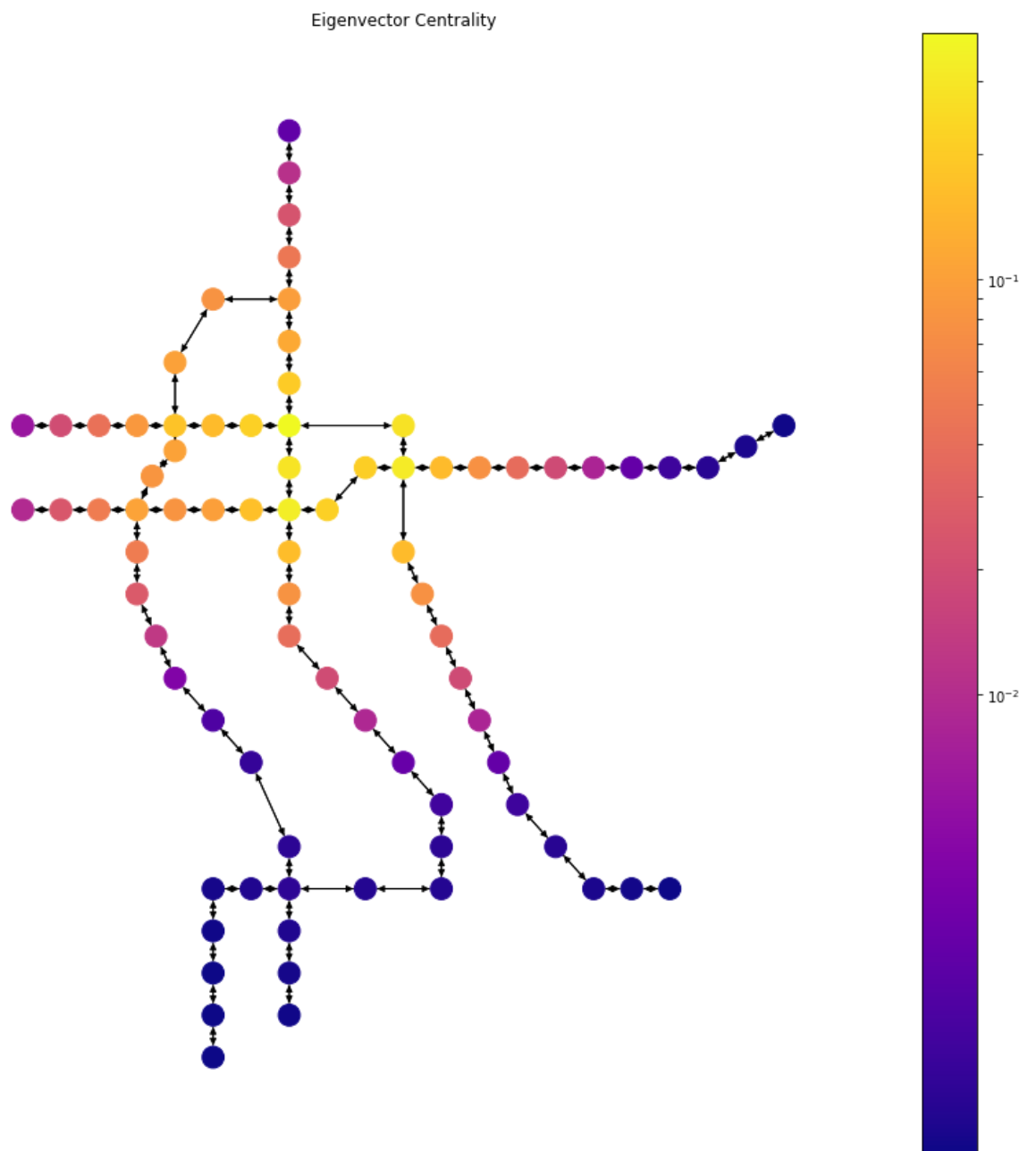
# 打印结果
print(centrality)
```

```
{'堰桥': 0.004316202741210824, '锡北运河': 0.010893833328162643, '西漳': 0.023179210592218946, '天一': 0.04760928150448337, '刘潭': 0.0969843598380947, '庄前': 0.1184374410}
```

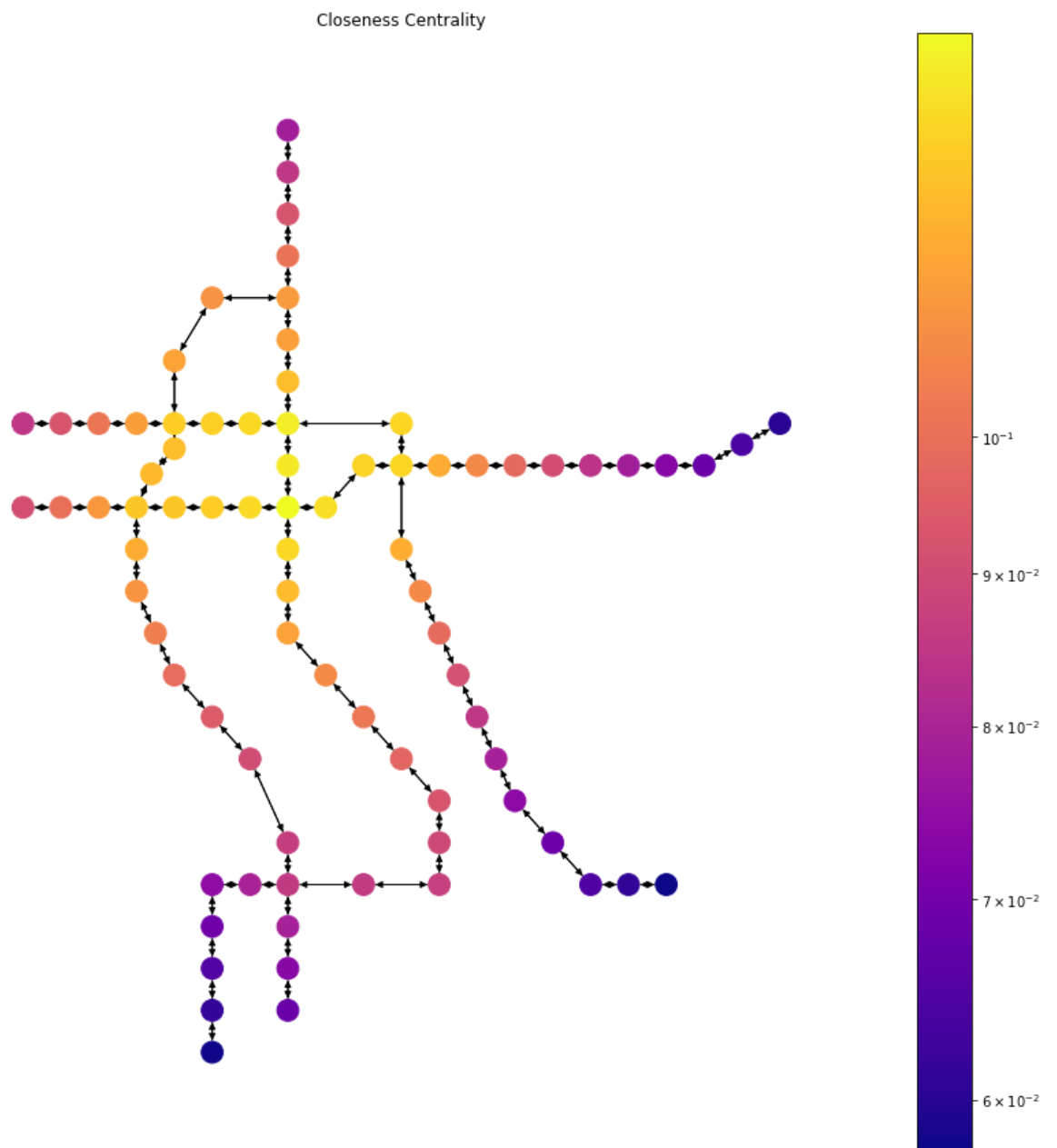
2602564, '民丰': 0.20194994444498202, '无锡火车站': 0.39128400387007983, '胜利门': 0.2861445028228152, '三阳广场': 0.3309464434115678, '南禅寺': 0.1628804272716254, '谈渡桥': 0.08016536032511595, '太湖广场': 0.03945786980816308, '清名桥': 0.01942679023234983, '人民医院': 0.009575525704825345, '华清大桥': 0.004741750193883773, '扬名': 0.0023922720985740794, '南湖家园': 0.001295552564676434, '塘铁桥': 0.0008762511680738678, '金匮公园': 0.0009135416889088395, '市民中心': 0.001425051268205751, '文化宫': 0.0007043192889653269, '江南大学': 0.0003481695535734714, '长广溪': 0.00017188698037136867, '雪浪': 8.420277553395554e-05, '葛埭桥': 3.981465955472527e-05, '南方泉': 1.5849780929973855e-05, '梅园': 0.009744294484032483, '荣巷': 0.024592482633322074, '小桃源': 0.052322148984763034, '河埭口': 0.10745900659447065, '大王基': 0.0823222007789482, '梁溪大桥': 0.10031299913443283, '五爱广场': 0.17086369949572155, '东林广场': 0.21542285714211254, '上马墩': 0.21278394393121644, '靖海': 0.3216481868083124, '广益': 0.1583015503334327, '柏庄': 0.07790893303170779, '东亭': 0.03834290097682587, '庄桥': 0.018869868516697794, '云林': 0.00928539986800214, '九里河公园': 0.004566908626820707, '查桥': 0.0022417197402646184, '映月湖公园': 0.0010913281848575593, '迎宾广场': 0.0005128678658984648, '无锡东站': 0.00020318976917900756, '苏庙': 0.00792532077199364, '钱桥': 0.020002408107007683, '龙山梢': 0.04255815908810195, '石门路': 0.08740935709398004, '盛岸': 0.17805378914196912, '吴桥': 0.1565411071394194, '北栅口': 0.2170475902273219, '广瑞': 0.28245984673986335, '东风': 0.15830161344460467, '叙丰': 0.07790909234421084, '太湖花园': 0.03834324001716409, '新光路': 0.018870565031890815, '旺庄路': 0.009286818996261749, '黄山路': 0.004569794286919273, '高浪东路': 0.0022475845471710662, '周泾巷': 0.0011032462424040805, '无锡新区': 0.000537085922029208, '长江南路': 0.0002524010894204549, '硕放机场': 9.999666400842556e-05, '广石路': 0.07873765531251427, '黄巷': 0.10174367564840633, '惠山古镇': 0.10369676693833664, '青山湾荣院': 0.08366426592108213, '西园弄': 0.052900256571951465, '体育中心': 0.02605131483412717, '夏家边': 0.012847551533089878, '蠡湖大桥': 0.00637239821458764, '大剧院': 0.0032338578752696993, '五湖大道': 0.0017874625434788924, '周新苑': 0.0012746431551826475, '吴都路': 0.0006965355679942142, '丰润道': 0.0003286303426738019, '博览中心': 0.00013060082621230216}

In [25]:

```
plt.figure(figsize=(15,15))
draw(G, pos, centrality, 'Eigenvector Centrality')
```



```
In [26]: plt.figure(figsize=(15,15))
draw(G, pos, nx.closeness centrality(G), 'Closeness Centrality')
```



```
In [27]: plt.figure(figsize=(15,15))
draw(G, pos, nx.betweenness centrality(G), 'Betweenness Centrality')
```

Betweenness Centrality

